Rapport TP – Résolution de Labyrinthe Comparaison de Dijkstra et A* avec variantes bidirectionnelles

Jérémy Doublet

3 avril 2026

Table des matières

1 Objectif du TP **2**

2 Modélisation du labyrinthe **2**
2.1 Structure de la classe Maze 2
2.2 Gestion des voisins 2
2.3 Génération des obstacles 2

3 Explication des algorithmes **3**
3.1 Dijkstra classique 3
3.2 Dijkstra bidirectionnel 3
3.3 A* classique 3
3.4 A* bidirectionnel 3
3.5 Reconstruction du chemin 3

4 Visualisation des résultats **4**
4.1 Comparaison avec un labyrinthe de 100 par 100 4
4.2 Comparaison avec un labyrinthe de 50 par 50 5

5 Analyse expérimentale **5**
5.1 Résultats numériques 5
5.2 Interprétation 5

6 Analyse du comportement du code **6**

7 Conclusion **6**

1 Objectif du TP

L'objectif de ce travail pratique est d'implémenter et comparer plusieurs algorithmes de recherche de chemin sur une grille 2D représentant un labyrinthe.

Le labyrinthe est généré aléatoirement avec des obstacles, puis résolu à l'aide de quatre algorithmes :

- Dijkstra classique
- Dijkstra bidirectionnel
- A* classique
- A* bidirectionnel

L'objectif est d'analyser :

- le nombre de sommets explorés
- la qualité du chemin trouvé
- l'impact du taux d'obstacles

2 Modélisation du labyrinthe

Le labyrinthe est représenté par une matrice 2D de taille 100×100.

- 0 : case libre
- 1 : obstacle

Le point de départ est $(5, 5)$ et l'arrivée $(94, 94)$.

2.1 Structure de la classe Maze

Le projet repose sur une classe `Maze` qui contient :

- la taille de la grille
- la matrice des obstacles
- le départ
- l'arrivée
- les algorithmes de résolution

```
self.grid = [[0 for _ in range(width)] for _ in range(height)]
```

Cette structure orientée objet rend le code facilement maintenable.

2.2 Gestion des voisins

Les déplacements autorisés sont les 4 voisins directs :

```
directions = [(1,0), (-1,0), (0,1), (0,-1)]
```

Cela correspond à une distance de Manhattan.

2.3 Génération des obstacles

Les obstacles sont générés aléatoirement :

```
if random.random() < obstacle_prob:
    self.grid[y][x] = 1
```

Le départ et l'arrivée ne peuvent jamais devenir des obstacles.

3 Explication des algorithmes

3.1 Dijkstra classique

Dijkstra explore les cases par ordre de coût croissant.

```
open_set = [(0, self.start)]
```

La file de priorité `heapq` garantit de toujours traiter la case la plus prometteuse.

3.2 Dijkstra bidirectionnel

Deux recherches sont lancées simultanément :
— départ → arrivée
— arrivée → départ
L'algorithme s'arrête lorsque les deux zones explorées se rencontrent.

3.3 A* classique

A* utilise une heuristique.
La fonction utilisée dans le code est :

```
return abs(p1[0]-p2[0]) + abs(p1[1]-p2[1])
```

Il s'agit de la distance de Manhattan :

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

La priorité devient :

$$f(n) = g(n) + h(n)$$

Cette heuristique guide efficacement la recherche vers l'objectif.

3.4 A* bidirectionnel

Cette version combine :
— la puissance de l'heuristique
— la réduction de zone du bidirectionnel
C'est l'algorithme le plus performant observé dans nos tests.

3.5 Reconstruction du chemin

Le chemin est reconstruit grâce aux dictionnaires de parents :

```
while current in came_from:
    current = came_from[current]
```

Pour le bidirectionnel, les deux demi-chemins sont fusionnés au point de rencontre.

4 Visualisation des résultats

La fonction `plot_all_comparisons()` permet d'afficher :
- obstacles en noir
- zones explorées en bleu
- chemin final en rouge
- départ en vert
- arrivée en jaune

4.1 Comparaison avec un labyrinthe de 100 par 100

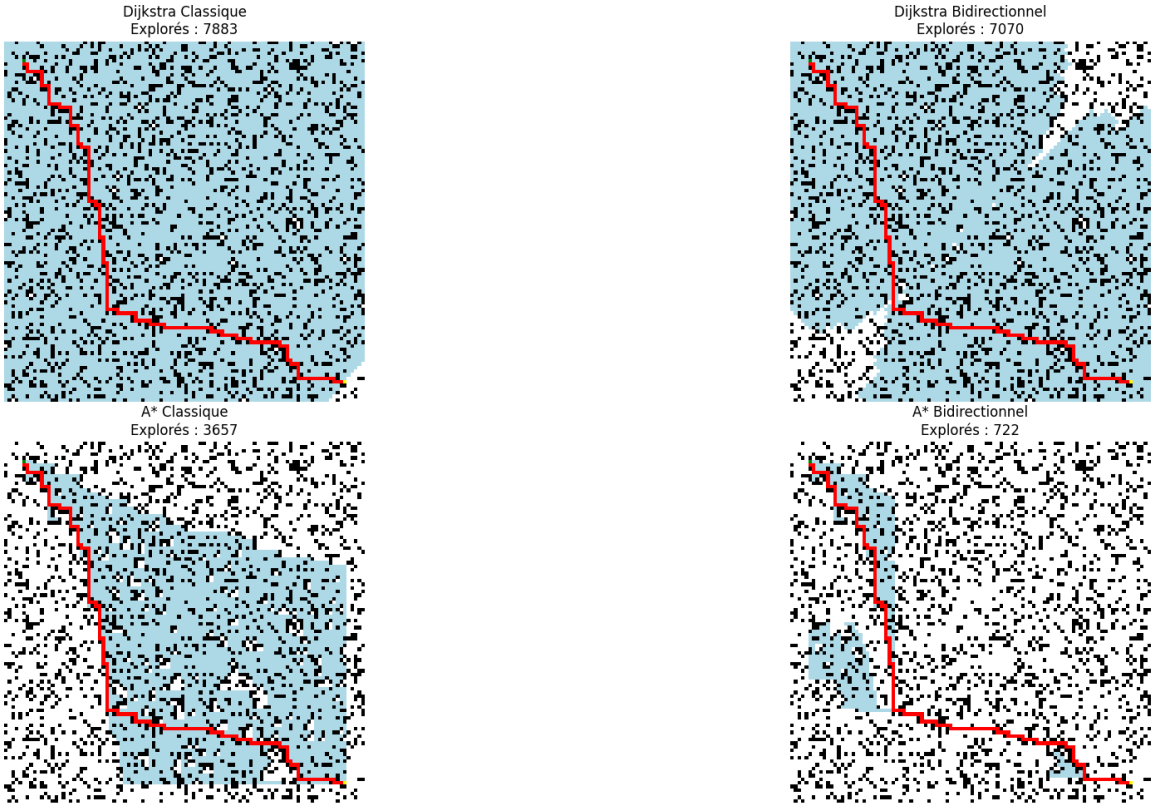

FIGURE 1 – Comparaison des quatre algorithmes avec un labyrinthe de 100 par 100

4.2 Comparaison avec un labyrinthe de 50 par 50

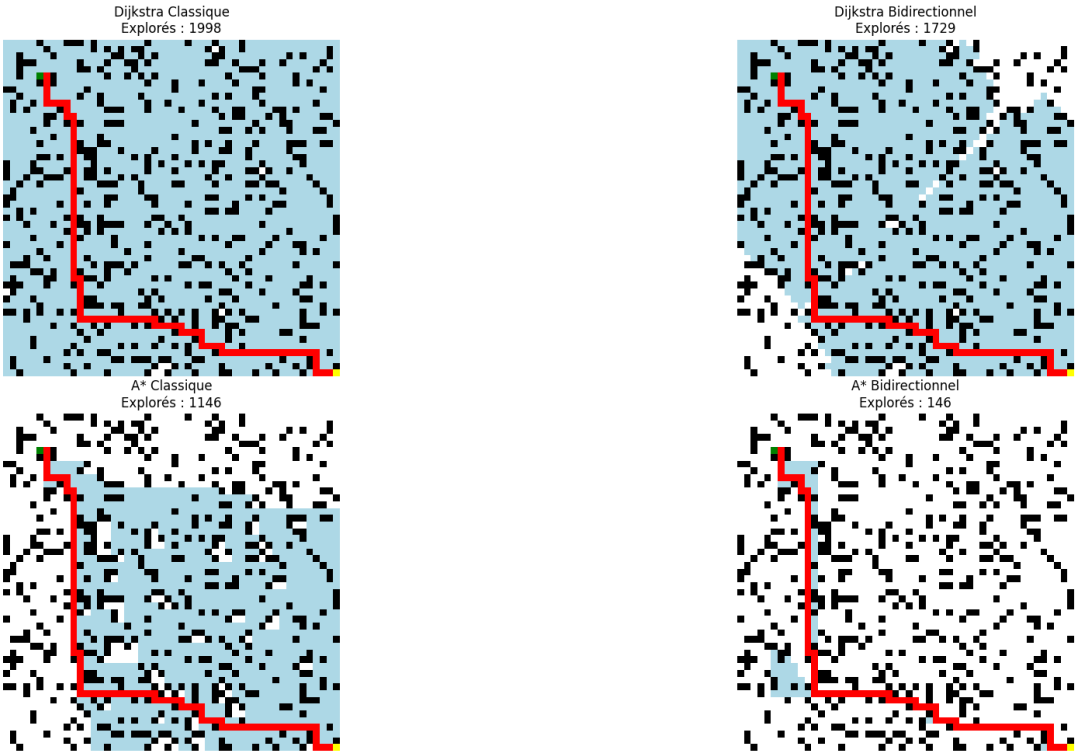

FIGURE 2 – Comparaison des quatre algorithmes avec un labyrinthe de 50 par 50

5 Analyse expérimentale

5.1 Résultats numériques

Algorithme	20% obstacles	10% obstacles
Dijkstra	7883	1998
Dijkstra Bi	7070	1729
A*	3657	1146
A* Bi	722	146

TABLE 1 – Nombre de sommets explorés

5.2 Interprétation

Les résultats montrent que :
— A* explore moins que Dijkstra
— le bidirectionnel améliore tous les cas
— A* bidirectionnel est systématiquement le meilleur
La baisse du taux d'obstacles réduit fortement le coût de recherche.

6 Analyse du comportement du code

Le code présente plusieurs points forts :
— architecture orientée objet
— modularité
— utilisation efficace de `heapq`
— heuristique adaptée à la grille
— visualisation claire des performances
Le projet peut facilement être étendu vers :
— déplacement diagonal
— labyrinthes pondérés
— animation en temps réel

7 Conclusion

Ce TP montre clairement l'intérêt des heuristiques et de la recherche bidirectionnelle. Les principaux enseignements sont :
— Dijkstra est robuste mais coûteux
— A* améliore fortement les performances
— le bidirectionnel réduit encore l'espace exploré
— A* bidirectionnel est le meilleur compromis
— Même si A* est généralement plus rapide sur une grille grâce à la distance de Manhattan, Dijkstra reste intéressant comme algorithme de référence, car il permet de mesurer l'apport réel de l'heuristique en comparant une recherche non guidée à une recherche orientée vers l'objectif.

Pour des labyrinthes de grande taille, A* bidirectionnel constitue la stratégie optimale.